Explainability in Reinforcement Learning

Maxime Alaarabiou

Wednesday 29th July, 2026

Compiled on 29/7/2026 at 14:28

Chapter 1

Latent Space Analysis

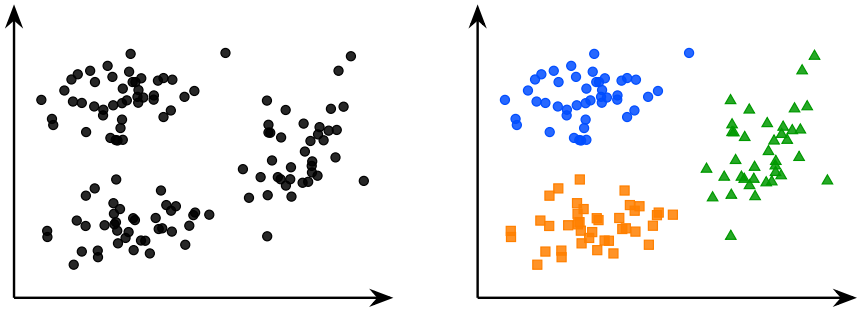
The ability of a Neural Network (NN) to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations play a central role in the network's ability to solve a task. Understanding their structure is therefore a key step toward understanding how NN achieve generalization.

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep Learning (DL) introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, NN gradually construct representations that become increasingly relevant to the task under consideration [Rumelhart et al., 1986]. These representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is precisely through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training. Since these representations are learned internally by the network and are not directly observable from the input or output spaces, they are commonly referred to as Latent Representation (LR). The collection of these LR learned throughout the network is commonly referred to as a Latent Space (LS).

These LS are now at the core of a growing number of practical applications. Neural video compression exploits the compactness of LS to encode information efficiently [Lu et al., 2019]. In Reinforcement Learning (RL), agent internal representations are used for efficient exploration in extremely open environments [Burda et al., 2018]. Recommendation systems model users and content within a shared LS to compute meaningful similarities [He et al., 2017]. Retrieval-augmented generation systems index and query knowledge bases through latent embeddings [Lewis et al., 2021]. Finally, Large Language Model (LLM) and multimodal architectures rely on the quality of learned representations to align heterogeneous modalities such as text, images, and audio [Radford et al., 2021]. These examples represent only some of the most widespread applications of LR.

Given their extensive use, the study of LR has become a central topic in DL research. A major objective is to understand how information is organized within these spaces, which concepts are encoded, and how these representations

Compiled on 29/7/2026 at 14:28



(a) Observed activations **(b)** Under the concept hypothesis

Figure 1.1: The same latent space activations seen without concept hypothesis **(a)** and under the working hypothesis that concepts emerge in the latent space **(b)**. Colors represent hypothetical concept memberships assigned by the model; they are not observed labels. Under this hypothesis the structure of the latent space becomes meaningful and amenable to analysis.

influence model behavior.

In this chapter, we first make explicit the working hypothesis that underlies much of the research on LS analysis. Throughout this manuscript, we refer to this assumption as the concept hypothesis 1.1. Once this conceptual framework has been established, we review the main approaches proposed in the literature to identify and extract concepts from LR. To the best of our knowledge, the main families of methods include probing methods (Section 1.2), semantic directions (Section 1.3), sparse autoencoders (Section 1.4), and clustering-based approaches (Section 1.5). Finally, Section 1.6 draws a comparative synthesis of these methods and discusses the open questions that remain.

1.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on LS analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, LS, concept, and semantics. The goal of this section is to make this conceptual framework explicit by defining its main components.

This hypothesis originates from the need to explain the generalization ability of deep NN observed after training, as discussed in Section **??**. A model is said to generalize when, for an input $x \in \mathcal{X}$ not seen during training, it still produces an output $\hat{y}$ close to the true output y. Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 1.1.1, whereby raw inputs are transformed into internal structures that are meaningful for the task [Bengio et al., 2014].

Definition 1.1.1: Abstraction

Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction or decision-making.

The ability of NN to perform abstraction arises from their intermediate activations. Indeed, a NN does not directly map x to y. Instead, as illustrated in Figure **??**, it computes a sequence of intermediate activations across hidden layers. These activations are numerical encodings of the input referred to as embeddings. Once training is complete, these embeddings have been structured so as to be capable of performing a specific task. To mark this distinction, we will refer to these trained embeddings as latent representations, as defined in Definition 1.1.2.

Definition 1.1.2: Latent Representation

A representation is the internal encoding of an input given by a intermediate layer of a trained NN.

Each hidden layer refines the representation of the previous layer. Each transformation progressively reshapes the representation into a form more directly aligned with the output space [Rumelhart et al., 1986]. This process can be described as a sequence of transformations where each $h^{(l)}$ is a representation:

$$x \to h^{(1)} \to h^{(2)} \to \cdots \to h^{(L-1)} \to \hat{y}$$

The set of all such representations produced by a layer over a dataset defines its LS, as defined in Definition 1.1.3.

Definition 1.1.3: Latent Space

The LS is the set of all representations produced by a layer over a dataset. It is shaped by learning and organizes representations according to task-relevant properties.

Under the concept hypothesis, the analysis of the LS is assumed to provide access to higher-level abstract structures learned by the model, as illustrated in Figure 1.1. These structures are referred to as concepts. In this manuscript, a concept is defined as an abstract representation that organizes information and supports reasoning (see Definition 1.1.4). Its relevance is therefore determined not by whether it is "true," but by whether it provides a useful description for the task under consideration.

Our definition of concepts is consistent with both the pragmatic tradition in philosophy and the information-processing view of cognition. Within these frameworks, concepts arise from the need to simplify an otherwise intractable reality. Because the environment contains more information than any cognitive system can process, intelligent behavior relies on abstractions, called concepts, that discard irrelevant details while preserving the information required to achieve a given objective [Peirce, 1878, Misak, 2013]. These concepts are realized as internal representations that reduce complexity and enable reasoning under limited computational resources [Newell, 1980, Simon, 1996]. Although these

theories were originally developed to explain human cognition, the concept hypothesis assumes that the same principles can be extended to deep NN.

In order to influence the model's computations, concepts must necessarily admit a concrete realization within the LS. We refer to this realization as the concept structure, or simply the structure. The definition of the structure is provided in Definition 1.1.5. The methods reviewed in this chapter all subscribe to the concept hypothesis, but they disagree on what structure a concept should take. This choice necessarily affects the extraction procedure, which is why several families of LS analysis methods coexist in the literature.

Definition 1.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.

Definition 1.1.5: Concept Structure

A concept structure is the specific form for how a concept is instantiated within the LS.

Now that the definition of a concept has been established, we can consider its relationship to human. According to our definition, we can imagine concepts that are useful for the machine but do not make sense to humans. This relationship is captured by the notion of semantics [Marconato et al., 2023], defined in Definition 1.1.6. Some concepts developed in LS can be readily associated with human concepts and are said to have high semantic value. Others may contribute to task performance while remaining difficult for humans to understand, and are said to have low semantic value.

Definition 1.1.6: Semantics

Semantics refers to the alignment between concepts in the model LS and concepts used by a human observer. A concept is semantic if it can be consistently associated with a human-interpretable concept.

It is important to note that this conceptual framework is not supported by a complete theoretical understanding of deep NN. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable structures within LS. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and LS analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing LS and extracting the concepts they encode. We begin with probing, one of the most direct approaches for assessing whether concepts are represented in the LS.

1.2 Probing

Probing methods aim to determine whether a given semantic concept is encoded within the LS of a NN [Li et al., 2024, Karvonen, 2024]. More specifically, they investigate whether the presence or absence of a selected concept can be inferred

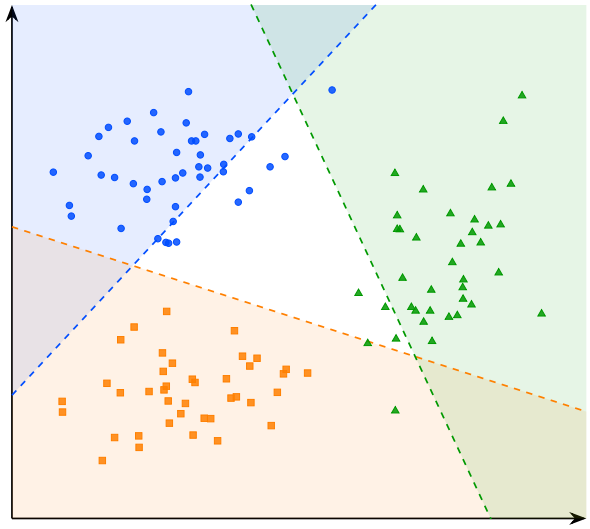


Figure 1.2: Decision boundaries of the linear probes for concepts A, B, and C. The colored regions indicate the areas classified positively by each probe.

from embeddings. The underlying assumption is that if a concept can be reliably decoded from these representations, then the network likely encodes information related to it in order to perform its task.

A probing analysis begins by identifying a set of semantic concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example x. The dataset can therefore be written as follows:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}, c_1^{(i)}, \ldots, c_n^{(i)})\}_{i=1}^{N}.$$

The data are subsequently projected into the LS of the pretrained model. We denote by h^l the representation vector obtained at layer l. From these LR, a classifier is trained in a supervised manner to predict the presence of the considered concept. This classifier is referred to as a probe and is denoted by p. Its purpose is to determine whether a given concept is present in the representation. Its purpose is therefore to assess whether information associated with the concept is accessible from the network's internal representations. This can be formalized as follows:

$$p_\theta^{l,j} : h^l \mapsto c_j, \quad \theta^* = \arg\min_\theta \mathcal{L}\big(p_\theta^{l,j}(h^l), c_j\big)$$

The classifier used to detect the presence or absence of a concept must remain intentionally simple. Indeed, if the probe has too much capacity, it may reconstruct the target information from complex correlations that do not necessarily correspond to information explicitly encoded in the LS. In such a case, it becomes difficult to determine whether the concept is genuinely represented by the network or merely reconstructed through the expressive power of the classifier. For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the NN and

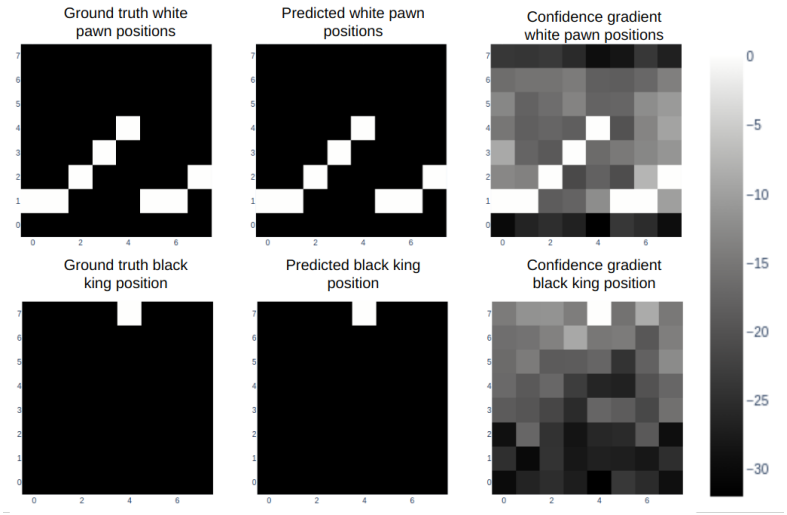


Figure 1.3: Probing analysis of a large language model trained to play chess [Karvonen, 2024]. Each row corresponds to a piece type: white pawns (top) and the black king (bottom). The left column shows the ground truth board positions, the middle column shows the positions predicted by the probe from the model's internal representations, and the right column shows the confidence gradient across board squares. The close agreement between ground truth and predicted positions suggests that the model internally maintains a structured representation of the board state, even though it was only trained on sequences of moves expressed in natural language.

potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the LS contains exploitable information related to the considered concept.

An application of this technique can be found in studies investigating whether LLM develop internal representations of their environment [Li et al., 2024, Karvonen, 2024]. In these works, researchers train LLM to play games such as chess or Othello in a purely textual setting. The model is provided only with sequences of moves and information about the actions it can take, all expressed in textual form. The central question is whether the LLM develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions. As illustrated in Figure 1.3, probes are able to reconstruct the full board state from the model's hidden representations, even though the only available information to the model is a sequence of text.

However, the fact that a concept can be decoded from LR does not necessarily imply that the model actually uses this information to perform its task. To assess whether representations play a causal role in the model's output, it is possible to perform interventions on the LR. An intervention consists of modifying a LR in a controlled manner by activating or deactivating a given concept, and then studying its impact on the model's output. If the resulting change in the model's output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, the LR h^l is minimally modified such that the probe prediction for the target concept c_j changes state, while the predictions associated with other concepts remain unchanged. This goal can be formulated as an optimization problem over the latent activation itself. The optimization is performed directly on the representation $h^{l'}$, using gradient-based optimization as introduced in Section **??**. The solution to this problem is the optimal modified representation h^{l*}, defined as:

$$h^{l*} = \arg\min_{h^{l'}} \underbrace{\gamma\, \mathcal{L}_{\text{flip}}\big(p_\theta^{l,j}(h^{l'}), c_j\big)}_{\text{flip target concept } c_j} + \underbrace{\lambda \sum_{k \neq j} \mathcal{L}_{\text{preserve}}\big(p_\theta^{l,k}(h^{l'}), c_k\big)}_{\text{preserve other concepts } c_{k\neq j}} + \underbrace{\beta \|h^{l'} - h^l\|_2^2}_{\text{minimality}}$$

where:

- h^l is the original LR of the input at layer l,
- $h^{l'}$ is the modified LR being optimized,
- h^{l*} is the optimal modified representation,
- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|h^{l'} - h^l\|_2^2$: regularization term ensuring the modified representation remains close to the original LR,
- γ, λ, β: weighting coefficients controlling the trade-off between the intervention strength, the preservation of other concepts, and the minimality of the perturbation.

In the context of LLM studied previously, such interventions have been used to remove specific pieces from the model's internal representation of the game state [Li et al., 2024, Karvonen, 2024]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. The LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model's behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias in what can be studied. Moreover, annotating a dataset for each concept is extremely time-consuming and requires significant human effort. Interventions in LS are also difficult to interpret, as it is not always clear what constitutes a meaningful change in the network's output under a given perturbation.

Moreover, probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a LR. However, this binary perspective can be overly restrictive, as concepts may be represented in a more continuous or graded manner, with varying degrees of presence. To obtain a more detailed characterization of these representations, other approaches have been proposed, such as the analysis of interpretable directions in LS.

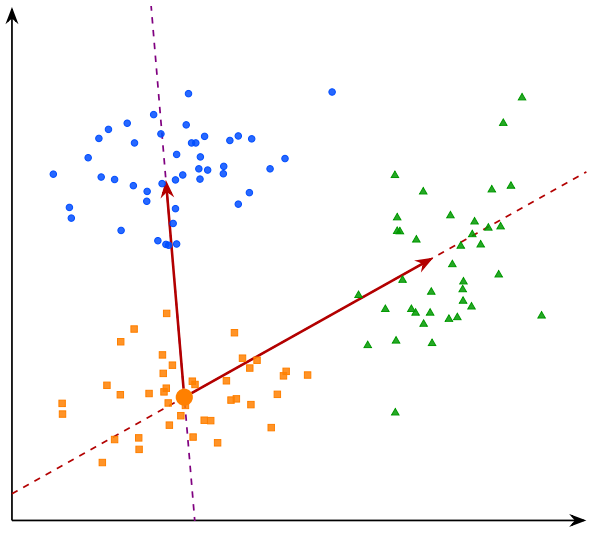


Figure 1.4: Interpretable directions in the latent space. The arrows represent semantic shifts from concept B toward concepts A and C. Moving along these directions in the latent space induces a coherent change in the model's behavior, suggesting that the transition between concepts is continuously encoded along these axes.

1.3 Interpretable Directions

To overcome the limitations of binary concept detection in probing methods, several approaches have been proposed to analyze the structure of LR. Among them, the notion of interpretable directions has emerged as a natural extension, aiming to capture the continuous nature of how concepts are encoded in representation spaces. Empirically, it has been observed that shifting a LR along a direction can induce coherent and semantically meaningful changes in the model's output [Haas et al., 2024, Voynov and Babenko, 2020]. Such transformations can be mathematically express as:

$$h' = h + \alpha \mathbf{v}_i,$$

where:

- h denotes the original embedding,
- $\mathbf{v}_i$ is a unit vector associated with concept i,
- α controls the strength of the intervention,
- h' is the resulting perturbed representation.

In many cases, the resulting change in the model's output varies smoothly with α, suggesting an approximately linear structure of the LS with respect to certain semantic factors, as illustrated in Figure 1.5. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the embedding space. Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be

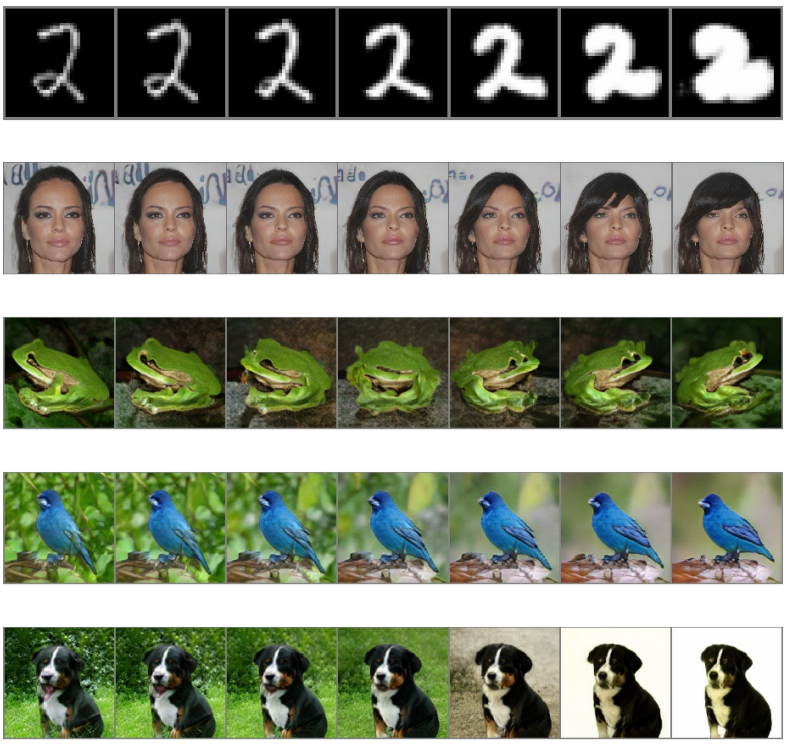


Figure 1.5: Interpretable directions in the latent space of a generative model [Voynov and Babenko, 2020]. Each row corresponds to a distinct direction $\mathbf{v}_i$, and each column to an increasing value of α. The resulting transformations are semantically coherent and diverse: stroke thickness for digits, head rotation for faces, texture and background for natural images.

associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in LS that align with meaningful variations in the encoded concepts.

This phenomenon has been extensively studied in generative image models. As illustrated in Figure 1.5, controlled perturbations of LR along interpretable directions can induce specific semantic transformations, such as stroke thickness modification in digit images, head rotation in facial images, or texture and background changes in natural images [Voynov and Babenko, 2020]. These observations have led to the development of the field of LS editing, whose objective is to manipulate generated content directly through modifications in LS rather than through changes in the input prompt.

However, a central challenge remains: identifying such interpretable directions is far from trivial. Most existing approaches rely on auxiliary models that encode human-defined semantics, such as CLIP [Haas et al., 2024], to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques 1.2, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised

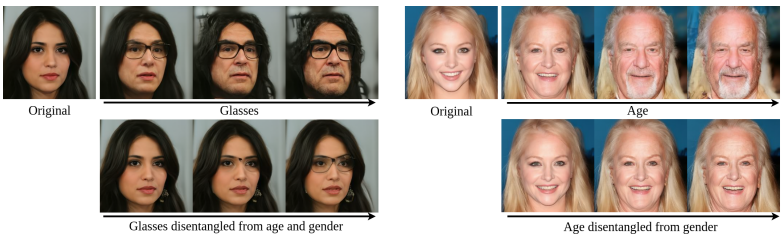


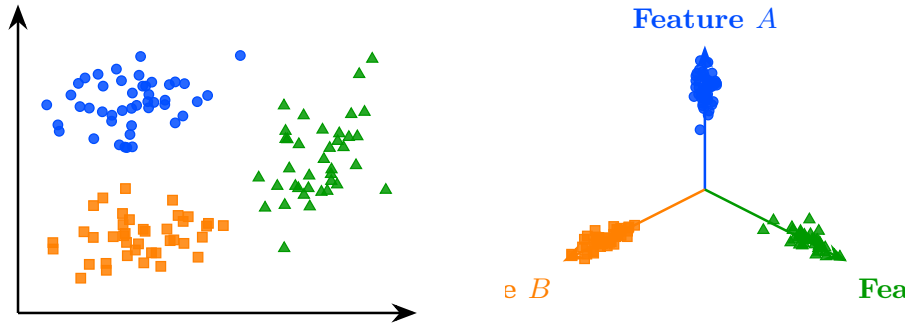
Figure 1.6: Illustration of entanglement in the latent space of a generative face model [Haas et al., 2024]. The top rows show entangled directions: moving along the glasses direction simultaneously modifies age and gender, and moving along the age direction also alters gender. This reflects the entanglement phenomenon, where a single direction in latent space induces unintended changes across multiple semantic concepts at once. The bottom rows show the corresponding disentangled directions, where each axis controls a single semantic attribute while leaving the others unchanged.

semantic discovery has emerged. Its goal is to uncover interpretable directions in LS without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in LS, while a second model attempts to recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent structure in LS [Voynov and Babenko, 2020]. In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations.

However, despite these advances, several challenges remain. It appears that not all concepts encoded by neural networks are linearly identifiable in LS. In practice, interpretable direction methods typically discover only a few dozen directions per dataset, which intuitively seems insufficient to account for the full diversity of semantic variations that a generative model is capable of producing.

In addition, LR often exhibit a phenomenon known as entanglement, illustrated in Figure 1.6, where multiple semantic attributes are mixed within the same directions of the representation space. Ideally, one would like each semantic factor to vary independently along a specific direction, without affecting other attributes. In practice, this is rarely the case: a single direction in LS often influences multiple semantic simultaneously, meaning that modifying one concept unintentionally alters several others. This lack of separability indicates that semantic information is not cleanly factored in the representation space, which motivates the search for more structured decompositions.

The limited number of linearly identifiable concepts, combined with the difficulty of isolating independent semantic factors, has motivated a stronger hypothesis about how neural networks organize information internally: the superposition hypothesis. In this context, a preferred analytical tool for investigating this hypothesis is the use of Sparse AutoEncoders (SAE), which can extract concepts from neural representations.



(a) Dense latent space **(b)** Sparse SAE space

Figure 1.7: Transformation from a dense latent space to a sparse SAE-style representation. On the left **(a)**, embeddings are entangled in the initial latent space. On the right **(b)**, the sparse representation aligns each concept with a dedicated feature axis.

1.4 Sparse Autoencoder

Approaches based on latent directions face inherent limitations. Empirically, only a restricted subset of concepts appears to admit a linear representation, and even these directions are often affected by significant entanglement with other concepts, illustrated in Figure 1.6.

To explain this phenomenon, a complementary hypothesis has been introduced, extending the assumptions underlying the Concept Hypothesis (1.1). This hypothesis, known as the Superposition Hypothesis [Elhage et al., 2022], posits that concepts are preferentially represented in a linear fashion, with each concept ideally corresponding to a distinct direction in LS. In this idealized setting, concept vectors would form an approximately orthogonal basis, where each direction independently controls a single factor of variation. Furthermore, only a small subset of the available concepts is expected to be active for a given representation. As a result, LR can be viewed as sparse combinations of concept directions. This idealized representation is referred to as a non-superposed representation and can be mathematically expressed as follows:

$$\mathbf{h} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i, \qquad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \qquad \|\boldsymbol{\alpha}\|_0 \ll n,$$

where:

- $\mathbf{h}$ denotes a LR,
- n is the total number of concepts in the dictionary,
- $\mathbf{v}_i$ represents the direction associated with concept c_i,
- α_i corresponds to the activation strength of concept c_i in the representation $\mathbf{h}$,
- $\mathbf{v}_i^\top \mathbf{v}_j \approx 0$ indicates that concept directions are approximately orthogonal, meaning each concept independently controls a single factor of variation,

Figure 1.8: A concept direction corresponding to the Golden Gate Bridge, discovered by a sparse autoencoder applied to a large language model [Templeton et al., 2024]. This direction activates strongly whenever the Golden Gate Bridge is evoked in the input, whether through English text, other languages, or even images, demonstrating that the learned concept is not tied to a specific surface form but captures a deeper semantic abstraction shared across modalities.

- $\|\boldsymbol{\alpha}\|_0 \ll n$ indicates that only a small fraction of concepts are active for a given representation.

According to the Superposition Hypothesis, latent representations do not perfectly follow this idealized structure because the representational capacity of NN is limited. For a given layer, the dimensionality of the representation space is finite, which constrains the number of concepts that can be encoded independently. As a result, the network cannot allocate a dedicated direction to every concept, and multiple concepts end up compressed into shared dimensions [Bricken et al., 2023]. This phenomenon is closely related to the entanglement observed in LS analyses: where entanglement is an empirical observation about the mixing of semantic attributes, the Superposition Hypothesis provides a theoretical explanation for why such mixing emerges.

Under this hypothesis, the phenomenon of superposition becomes an unavoidable obstacle for any method that attempts to analyze LR directly in the original activation space. Motivated by this view, and with the goal of recovering the idealized non-superposed, a large body of work has focused on developing methods to desuperpose LR. To this end, a common strategy consists in projecting LR into a higher-dimensional space while imposing additional constraints designed to recover a non-superposed representation. A widely used implementation of this idea is the SAE, which can be defined as follows:

$$z = f_{\text{enc}}(h), \quad \hat{h} = f_{\text{dec}}(z)$$

$$\mathcal{L}_{\text{auto}} = \gamma \|h - \hat{h}\|_2^2 + \lambda \|z\|_1$$

where:

- $f_{\text{enc}}(h)$ is the encoding function that projects the original representation h into a higher-dimensional sparse space,
- $f_{\text{dec}}(z)$ is the decoding function that reconstructs the original representation from the sparse code,

- $z \in \mathcal{Z}$ is the sparse representation of h, with $\dim(z) \gg \dim(h)$,
- $\hat{h}$ is the reconstructed representation,
- $\gamma\|h-\hat{h}\|_2^2$ is the reconstruction loss ensuring that the original representation can be faithfully recovered,
- $\lambda\|z\|_1$ is the sparsity regularization term that encourages most components of z to be zero,
- γ, λ are weighting coefficients controlling the trade-off between reconstruction fidelity and sparsity.

In practice, both f_{enc} and f_{dec} are implemented as linear projections into the high-dimensional space $\mathcal{Z}$. This choice is motivated by the same reasoning as in probing methods (Section 1.2). Using overly expressive projection functions would introduce additional bias, making it difficult to attribute the structure of the learned representations to the model itself rather than to the projection. The parameters of these projection functions are learned by minimizing $\mathcal{L}_{\text{auto}}$ using gradient-based optimization, as introduced in Section **??**. Under this formulation, each dimension of $\mathcal{Z}$ is interpreted as a dedicated concept direction. The corresponding basis vector serves as the support for a distinct concept encoded by the model.

The method proves to be powerful when successfully applied to real language models [Templeton et al., 2024]. In these applications, the idea is to desuperpose LLM LR using SAE in order to recover a non-superposed representation. Once this representation is obtained, it has been shown that many concept vectors exhibit strong semantic properties. A well-known example is the Golden Gate Bridge concept direction, illustrated in Figure 1.8, which activates systematically whenever the input text or image refers to the Golden Gate Bridge, regardless of the language or modality.

It is possible to artificially activate this concept direction by directly modifying the sparse representation z, even when the input does not contain any reference to it. This operation constitutes an explicit intervention on the model's latent representations. The edited representation is then projected back into the original activation space and the model proceeds with standard next-token generation from h^*:

$$h^* = f_{\text{dec}}(z^*), \quad \text{where} \quad z^* = z + \delta\, e_i,$$

where:

- z is the original sparse representation of the input,
- z^* is the modified sparse representation,
- e_i is the basis vector of $\mathcal{Z}$ associated with the target concept direction,
- δ controls the strength of the artificial activation,
- h^* is the resulting edited representation in the original activation space.

Empirically, it is observed that even when the prompt is unrelated to the Golden Gate Bridge, the model's output shifts to discussing it. This provides evidence for the causal role of these learned features. This type of intervention

has been used by Anthropic researchers to steer or suppress specific model behaviors, effectively enabling a form of behavior control through latent-space editing.

Despite their empirical success, SAE exhibit several important limitations that question some of their underlying assumptions. These limitations can be summarized as follows:

- **Instability of the decomposition.** Repeating the SAE training process multiple times on the same model does not yield the same decomposition of the LS. Different runs produce different concept directions in $\mathcal{Z}$, even when achieving comparable reconstruction performance, which raises questions about the uniqueness and reliability of the discovered structure.

- **Lack of semantic coverage.** In the high-dimensional space $\mathcal{Z}$, a large fraction of dimensions do not correspond to any interpretable semantic concept. This abundance of semantically vacuous directions questions the extent to which the decomposition genuinely reflects the conceptual structure of the model.

- **Sparsity–semantics trade-off.** Increasing the sparsity coefficient λ beyond a certain threshold degrades the semantic quality of the learned features [Jermyn et al., 2024]. The representations tend to lose meaningful structure, suggesting that enforcing excessive sparsity comes at the cost of interpretability.

- **Failure of scaling to remove superposition.** Under the Superposition Hypothesis, increasing the dimensionality of the representation space should reduce superposition. However, this is not observed in practice. Even in very high-dimensional LS, representations remain partially superposed, indicating that scaling alone does not resolve the entanglement of features.

These limitations suggest that SAE, while powerful, do not fully resolve the problem of representational structure, and that convincing empirical results do not necessarily validate the underlying theoretical assumptions. Indeed, probing (Section 1.2), interpretable directions (Section 1.3), and SAE all rest on a shared assumption: that semantic structure can be recovered through approximately linear decompositions of LR. This assumption has driven significant empirical progress, yet it remains theoretically fragile. The following section therefore introduces a complementary approach that relaxes this constraint entirely, requiring no assumption about the linear organization of concepts: deep clustering.

1.5 Clustering

The LS analysis methods presented so far all rely, either explicitly or implicitly, on assumptions about the linear organization of concepts within the LS. For probing methods, concepts must be at least partially linearly separable for simple probes to succeed 1.2. Interpretability through latent directions assumes that variations associated with a concept can be captured by moving along a linear axis of the LS 1.3. Similarly, the superposition hypothesis assumes that concepts are represented through linear directions that become entangled because of representational capacity constraints 1.4. Although these assumptions have led

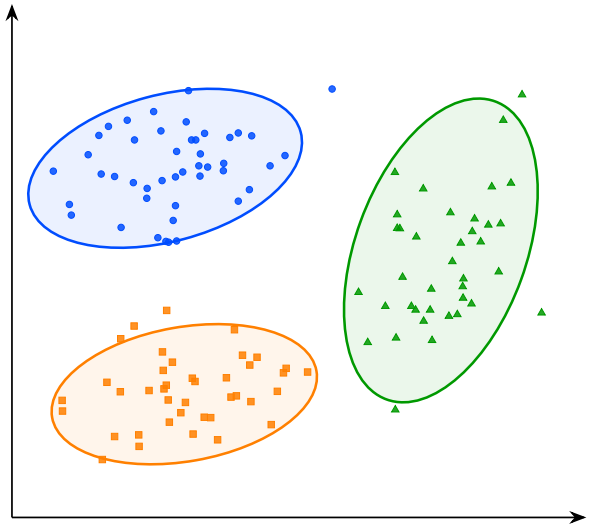


Figure 1.9: Visualization of the latent space structure. The points represent embeddings associated with concepts A, B, and C. The ellipses indicate the approximate extent of each group in the latent space and highlight their geometric separation.

to numerous empirical successes, their theoretical justification remains limited. More importantly, several observations suggest that the linearity hypothesis may not fully capture the structure of LR.

For instance, some concepts can only be recovered using non-linear probes. Likewise, methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in sparse autoencoder approaches, enforcing excessive sparsity often degrades reconstruction quality and interpretability, suggesting that the underlying representation may not be perfectly described by a sparse linear decomposition. Taken together, these observations indicate that semantic concepts may not always be organized according to a simple linear structure. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of LS.

To the best of our knowledge, clustering-based analysis is the only family of LS analysis methods that does not rely on any assumption of linear structure, which is why the following section is devoted to its presentation [Ren et al., , Wei et al.,]. Rather than assuming that concepts correspond to directions or linear subspaces, clustering methods only assume that semantically similar inputs produce nearby LR. Conversely, inputs that differ significantly from a semantic perspective are expected to be located farther apart in the LS. This is a considerably weaker assumption, in the sense that it is empirically verified in a wide range of settings.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of embeddings that occupy the same region of the LS. Concepts are therefore no longer associated with directions, but with clusters of neighboring representations, as illustrated in Figure 1.10. This idea forms the basis of the research field known as deep clustering. In these approaches, a collection of inputs is first projected into a LS using a NN. The resulting embeddings form a point cloud on which standard clustering algorithms can

Figure 1.10: Clusters discovered by a deep clustering model applied to natural images [Xie et al.,]. Each row contains the ten highest-scoring elements of a single cluster. The visual coherence within each row, grouping animals, vehicles, and aircraft into distinct clusters, suggests that the learned latent representations capture semantically meaningful structure without any explicit supervision.

be applied. The obtained clusters are then interpreted as representing distinct semantic concepts.

Deep clustering has been particularly successful in unsupervised image classification. Typically, an autoencoder is composed of an encoder f_{enc} and a decoder f_{dec}. The encoder maps an input $x^{(i)} \in \mathcal{X}$ to a low-dimensional LR $z^{(i)} \in \mathbb{R}^d$:

$$z^{(i)} = f_{\text{enc}}(x^{(i)}), \quad d \ll \dim(\mathcal{X}).$$

To ensure that this compressed representation does not discard essential information, the decoder reconstructs the input from the LS:

$$\hat{x}^{(i)} = f_{\text{dec}}(z^{(i)}) = f_{\text{dec}}(f_{\text{enc}}(x^{(i)})).$$

The model is trained to minimize the reconstruction error between the original input and its reconstruction, typically using a Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{rec}} = \frac{1}{N} \sum_{i=1}^{N} \left\| x^{(i)} - \hat{x}^{(i)} \right\|_2^2.$$

This objective encourages the LS to preserve the most informative features of the input while enforcing a compact representation.

Clustering algorithms are then applied to the learned embeddings.

Let $\mathcal{D} = \{x^{(i)}\}_{i=1}^{N}$ denote a dataset of inputs. Through the encoder f_{enc}, this dataset induces a point cloud in the LS:

$$\mathcal{Z} = \{z^{(i)} \in \mathbb{R}^d \mid z^{(i)} = f_{\text{enc}}(x^{(i)}),\ x^{(i)} \in \mathcal{D}\}.$$

Clustering algorithms are then applied to this set $\mathcal{Z}$ of embeddings in order to identify groups of semantically similar representations.

An example of this methodology can be observed in Figure 1.10, where the approach is applied to the STL-10 dataset [Coates et al., 2011] using k-means clustering [MacQueen, 1967] with a number of clusters arbitrarily fixed to $K = 10$ [Xie et al.,]. For each cluster, the corresponding line shows the 10 samples closest to the associated centroid in the LS.

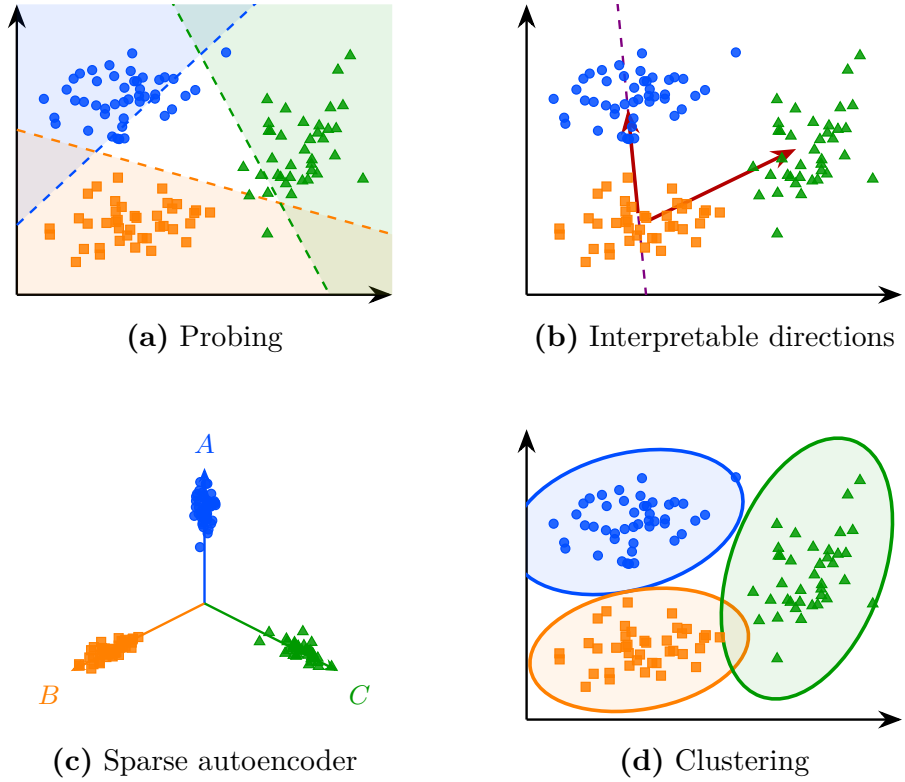
We observe that the method captures coherent semantic categories such as animals, vehicles, and flying entities. This result is particularly striking given that the only available information to the system consists of raw images. It suggests that the compression process induces a LS in which semantically meaningful structures naturally emerge, and that these structures correspond to high-level concepts.

More advanced approaches combine representation learning and clustering objectives within a single optimization process. In this setting, additional loss terms encourage embeddings to organize themselves around cluster centroids during training. This often produces LS that are easier to analyze and improves the quality of the discovered semantic structure [Guo et al., , Miklautz et al.,].

Despite its effectiveness, clustering-based analysis presents several important limitations. First, as with most unsupervised semantic discovery techniques, the interpretation of the resulting clusters remains partially subjective: while benchmark datasets often provide class labels that facilitate evaluation, there is no guarantee that the discovered clusters correspond to the categories that humans consider most relevant, and alternative semantic groupings may be equally valid. Second, clustering methods are highly sensitive to design choices, as the number of clusters, the distance metric, and the clustering algorithm itself can all significantly influence the results, making parameter selection challenging and often requiring substantial empirical tuning. Finally, to the best of our knowledge, no intervention technique has yet been proposed in the context of clustering-based analysis: unlike probing or SAE, which allow targeted modifications of LR to study their causal role, clustering methods currently offer no mechanism to alter the model's behavior by acting directly on the discovered structure.

1.6 Conclusion

The four families of methods reviewed in this chapter, namely probing, interpretable directions, sparse autoencoders, and clustering, are summarized in Figure 1.11 and all build upon the concept hypothesis introduced in Section 1.1. Each of them approaches the question from a different angle, but they share a common objective: to uncover how LR are structured and what semantic information can be extracted from them.



(a) Probing **(b)** Interpretable directions

(c) Sparse autoencoder **(d)** Clustering

Figure 1.11: Overview of the four latent space analysis methods studied in this chapter, applied to the same point cloud.

However, a fundamental difficulty remains. There is currently no consensus on how to rigorously evaluate to what extent these methods genuinely provide access to the internal concepts of a model. Most existing metrics assess reconstruction quality or linear separability, but these do not directly measure whether the discovered structures correspond to meaningful abstractions from the model's perspective. This leaves open the question of what it means, formally, for a concept to be represented in a NN.

Addressing this question may require contributions from disciplines beyond computer science and mathematics. As the notion of semantics is inherently tied to human interpretation, a purely formal or computational account may be insufficient. Insights from philosophy, cognitive science, or psychology could prove valuable in grounding the notion of concept in a way that is both theoretically rigorous and empirically tractable. Despite these open questions, the methods studied in this chapter provide a practical foundation for analyzing the internal representations of NNs.

Chapter 2

Explainability in Reinforcement Learning

Today, the majority of Artificial Intelligence (AI) agents with which humans interact are deployed in controlled or low-risk environments. For example, agents capable of playing games such as Go operate in settings where potential errors remain limited in terms of consequences [Silver et al., 2017]. Even more autonomous systems, such as agents based on Large Language Models (LLMs), remain constrained by human supervision mechanisms and are often restricted to tasks such as code generation or information retrieval [Chen et al., 2021, Lewis et al., 2021]. Safety-critical applications integrating machine learning components are still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees [Seshia et al., 2022]. These requirements include in particular:

- validating the system's functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL) [Mnih et al., 2013]. As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. To address this limitation, a research field has emerged: eXplainable Reinforcement Learning (XRL), a subfield of eXplainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents.

This chapter presents the main existing XRL methods. We first define what is meant by an explanation in the context of XAI. We then propose a taxonomy for organizing existing XRL methods. This taxonomy serves as the basis for the state-of-the-art review presented in the remainder of the chapter.

2.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in general, along with the explainability method used to produce it. We then describe how this concept applies to XAI. Finally, we further detail the properties that characterize an explainability method in the context of XAI.

2.1.1 Explanation

Establishing what is meant by an explanation is challenging due to the fundamentally multidisciplinary nature of the concept. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely computational or mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI [Miller, 2019, Bellucci et al., 2021].

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 2.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this manuscript (Definition 2.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning. An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 2.1.4). This refinement is obtained through a transformation process that produces an explanation from the object being explained. We refer to this process as an explainability method (Definition 2.1.5), detailed further in Section 2.1.3.

For an observer to understand an explanation, the information it contains must be interpretable (Definition 2.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 2.1.1: Object

The object of an explanation is the system the observer wants to understand.

Definition 2.1.2: Observer

The observer is the human who aims to understand an object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 2.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

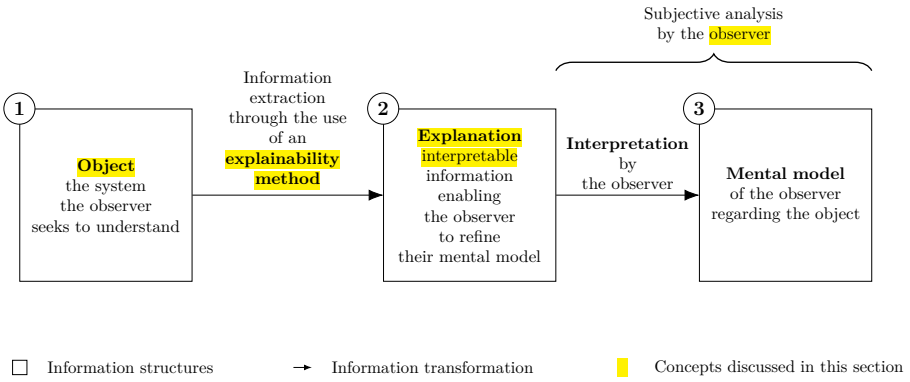


Figure 2.1: Explanation process.

Definition 2.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

Definition 2.1.5: Explainability Method

An explainability method is a transformation process that produces an explanation from the object being explained.

This definition of explanation provides the foundation for the approach to explainability adopted throughout this manuscript. Its generality allow it to encompass the different explainability methods considered in the following sections.

Moreover, this perspective avoids a binary interpretation of explainability, in which a system is considered either explainable or non-explainable. The notion of a non-explainable system is difficult to justify from a scientific standpoint. Scientific inquiry is driven by the assumption that observed phenomena can be investigated through the development of increasingly accurate analytical tools. Declaring a system to be inherently impossible to explain would therefore conflict with this methodological principle. This approach aligns with the philosophical Principle of Sufficient Reason, which states that every phenomenon rests upon an explainable basis [Melamed and Lin, 2023].

This quantitative perspective allows us to express an intuitive idea: not all explanations provide the same level of value [Doshi-Velez and Kim, 2017]. The effectiveness of an explanation depends on its ability to improve the observer's understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer's misunderstanding of a key mechanism.

Figure 2.1 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer, as a chain of three information structures, depicted as boxes and connected by arrows denoting a transformation of information:

(1) The **object** of the explanation, that is, the system the observer seeks to

understand.

(2) The **explanation**, the interpretable information produced from the object. This transformation is performed through the use of an explainability method.

(3) The **mental model** held by the observer regarding the object.

A brace spanning boxes (2) and (3) indicates that this second half of the process, from the explanation to the resulting mental model, constitutes a subjective analysis carried out by the observer.

2.1.2 Explainable Artificial Intelligence

Now that the general framework of explanation has been established, we can examine how this concept applies in the context of AI. In the context of XAI, the object of explanation is an AI system. More precisely, XAI emerged as a response to the success of DL models [Gunning and Aha, 2019, Barredo Arrieta et al., 2020]. In these architectures, information is represented in a distributed manner across the network through patterns of activation and connections between computational units. Unlike symbolic approaches, individual components of Neural Networks (NNs) cannot be directly associated with human-defined concepts. From this starting point, we identify two types of approaches:

- **Substitutive explainability:** In this approach, the underlying assumption is that there is nothing to explain within a NN. There is no semantic meaning associated with each computational component, and therefore nothing to interpret. This corresponds to the so-called "black box" view of NNs. Under this assumption, XAI consists of replacing as much of the DL system as possible with alternative AI methods [Rudin, 2019].

- **Intrinsic explainability:** In this approach, it is argued that NNs inherently contain mechanisms that can be explained. The limitation is considered to lie not in the networks themselves, but in the absence of adequate tools to analyze their functioning [Olah et al., 2020].

This manuscript considers methods from both approaches in order to provide a comprehensive overview of the XAI field. According to the definition of explanation adopted in this manuscript (Definition 2.1.4), we consider that any system can be explained. As a consequence, the perspective adopted throughout this manuscript is that of intrinsic explainability.

2.1.3 Explainability Method in XAI

Now that the general framework of XAI has been established, we need to further detail the properties of an explanation in this context. In our framework, an explanation in the field of XAI can be characterized by three components:

- **Source:** refers to the origin of the information used by the explainability method. The explainability method transforms the raw information produced by the AI system into an explanation that can be understood by an observer.

- **Form:** refers to the representation through which the explanation is communicated. The form defines how the information is presented to the observer and must allow human interpretation. An explanation can take many different forms, including text, images, computer code, and mathematical equations.

- **Subject:** refers to the aspect of the AI system that is being explained. In XAI, this distinction is commonly associated with the difference between local and global explanations. A local explanation describes why an AI system produced a specific output in a given situation. A global explanation aims to describe the general functioning of the AI system.

These three properties of an explanation are entirely determined by the explainability method (Definition 2.1.5) used to produce it. In this context, an explainability method is a transformation process that maps a source of information into an explanation characterized by a specific form and subject. Research in XAI focuses on developing new methods capable of performing this transformation in order to overcome the lack of interpretability associated with DL models.

2.2 Taxonomy of XRL Methods

Now that the concept of XAI has been defined, we turn to its application in RL, known as XRL. As in XAI, explanations in XRL are produced by explainability methods and can be described in terms of their subject, source, and form. This section reviews the possible variations of these three aspects in the specific context of RL.

2.2.1 Subject

The first question to address is the subject of explanation in the context of RL. To answer this question, it is necessary to identify the different components involved in a RL system and determine which of them constitutes the object of explanation. A RL system can be described through two main components: the environment and the policy.

The environment defines the dynamics in which the agent evolves, including the possible states, observations, actions, and transitions. Environments are simulations designed by humans, and their dynamics are explicitly defined. Therefore, it is assumed that the environment is known to the observer and does not require explanation.

The main object of interest is therefore the agent's policy. The policy represents the decision-making mechanism of the agent. It is a function that maps observations to actions and is learned through interaction with the environment in order to maximize the expected cumulative reward. The policy is the component that encodes the learning process and that XRL aims to explain.

Explaining a policy introduces additional challenges compared with classical XAI settings. In traditional XAI problems, the objective is to explain an isolated decision, such as determining which features of an image led to a classification result. In contrast, RL introduces a temporal dimension into the decision-making process. An action is not only determined by the current observation but also

influences future states and rewards. Therefore, understanding the behavior of an agent may require considering a sequence of interactions between the agent and its environment.

This temporal dimension leads to a distinction between two objectives of explanation in XRL:

- **Decision explanations:** focus on identifying the elements of the current observation that influenced the action selected by the policy. This type of explanation is close to classical XAI approaches, as it analyzes the relationship between inputs and outputs while abstracting away the sequential nature of the decision process.

- **Strategy explanations:** aim to characterize the agent's behavior over time. They focus on sequences of decisions and interactions with the environment in order to understand how the agent achieves its long-term objective. This type of explanation is specific to RL, as it addresses the temporal and goal-oriented nature of the learning process.

These two objectives should not be considered as strictly separated categories, but rather as two extremes of a continuum. Indeed, understanding the elements of an observation that influence a specific decision can help the observer generalize this information to infer broader aspects of the agent's behavior. Conversely, understanding the agent's overall strategy can provide insights into the reasoning behind individual decisions.

2.2.2 Source

We identify three sources of explanation in RL, all obtained during or after the learning process and containing information about the agent:

- **Policies:** policies are functions that generate a distribution over the action space for a given observation. The objective of RL is to learn a policy that maximizes the expected cumulative future rewards. During the learning phase, the policy is iteratively improved through interactions with the environment.

- **Trajectories:** trajectories represent sequences of successive interactions between the agent and the environment. At each step, the agent selects an action based on its observation, which modifies the environment and produces a reward as well as a new observation. This process continues until the end of the trajectory.

- **Value functions:** value functions provide predictions about the future outcome of a trajectory from a given observation. The most commonly used value function is the critic function, which estimates the expected sum of future rewards. This prediction guides the learning process, as the agent updates its policy to maximize the estimated future return. However, other types of value functions can also be used as sources of explainability.

2.2.3 Form

We identify six forms of explanation in the RL setting:

Explanation Subject	Explanation Form	Explainability Method Family	Source of Explanation		
			Policy	Trajectories	Value functions
Decision	Policy Model	*Interpretable policy learning*	✓		
		Policy approximation via interpretable model	✓		
	Saliency Map	*Observation perturbation*	✓		
		Observation gradient analysis	✓		
		Attention-based architecture	✓		
	Counterfactual	*Latent space usage for counterfactual generation*	✓		
Strategy	Agent Trajectory	*Hierarchical decomposition*	✓	✓	
		Extraction via critic analysis		✓	✓
	Trajectory Prediction	*Critic enrichment*			✓
	Interaction Model	*Bayesian network learning*		✓	
		Latent space usage for MDP generation	✓		

Table 2.1: Taxonomy of explainability methods in reinforcement learning.

- **Policy Model:** an interpretable model of the agent's policy function.
- **Saliency Map:** a weighting of the input features representing their influence on the output of the policy function.
- **Counterfactual:** a modified observation obtained by perturbing the original observation in order to produce a different action from the agent.
- **Agent Trajectory:** a sequence of successive interactions between the agent and the environment.
- **Trajectory Prediction:** a prediction about the future evolution of the trajectory produced by a value function.
- **Interaction Model:** an interpretable model of the interactions between the agent and the environment.

Among the three components characterizing an explanation, the form is the one that varies most significantly across explainability methods. While subjects and sources tend to be relatively stable in the XRL literature, the diversity in how explanations are presented is considerably greater. The list above is by no means definitive and may evolve as new approaches emerge. For this reason, the following section is organized according to the form of explanations and presents existing explainability methods based on this criterion.

2.3 State-of-the-Art Review of XRL Methods

The purpose of this section is to detail the functioning of existing explainability methods (see Table 2.1). They are presented according to the form of explanation they produce, following the taxonomy introduced in Section 2.2.

Each subsection introduces one of the six forms of explanation identified in Section 2.2.3. Within each subsection, each subsubsection is dedicated to a family of methods capable of producing that form of explanation.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation or in the details of the generation process, while relying on the same underlying principle.

2.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NNs is the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [Topin et al., 2021], algorithms [Trivedi et al., 2022], formal logic [Payani and Fekri, 2019], and linear models [Garreau and von Luxburg, 2020].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently simple or well structured, the observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent's strategy.

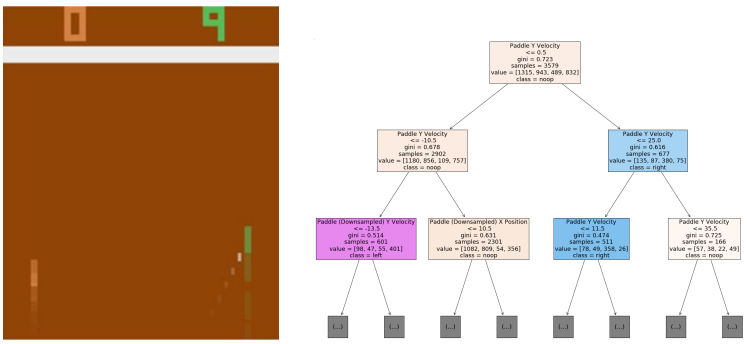
The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or conditions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

To obtain an interpretable policy model, two families of explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This family of methods aims at learning a policy represented by an **interpretable** model [Topin et al., 2021, Trivedi et al., 2022]. It is the most direct approach to addressing the challenges posed by XRL. This type of training requires modifying the Markov Decision Processes (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a **reduction** requires the



(a) (b)

Figure 2.2: Policy approximation via an interpretable model [Sieusahai and Guzdial, 2021]. (a) shows an observation from the Atari game "Pong", and (b) shows the decision tree obtained by approximating a deep reinforcement learning policy trained on this environment.

integration of **expert knowledge** into the learning process. With this family of methods, the *interpretable policy function* itself is used to provide explanations to the observer.

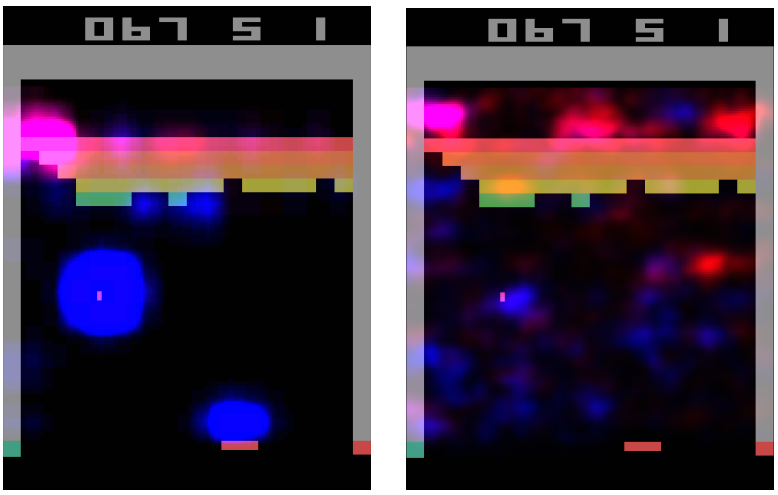
Policy Approximation via Interpretable Model

An interpretable model can be trained to approximate a policy learned through deep RL [Sieusahai and Guzdial, 2021]. In this setting, the problem is formulated as a supervised learning task, where the original deep RL policy is used to label each observation with the action it selects (see Figure 2.2a). Using this dataset, the interpretable model is trained to reproduce the behavior of the original policy, yielding for instance the decision tree illustrated in Figure 2.2b. The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment in place of the original policy, thereby providing a high degree of control over the agent.

2.3.2 Saliency Map

The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 2.3). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 2.3b, interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.



(a) (b)

Figure 2.3: Heatmaps obtained by [Greydanus et al., 2018] for the same observation of the Atari game "Breakout". Highlighted areas correspond to the most influential regions for the policy function's decision. (a) is obtained via observation perturbation (see Section 2.3.2), whereas (b) is obtained via observation gradient analysis (see Section 2.3.2).

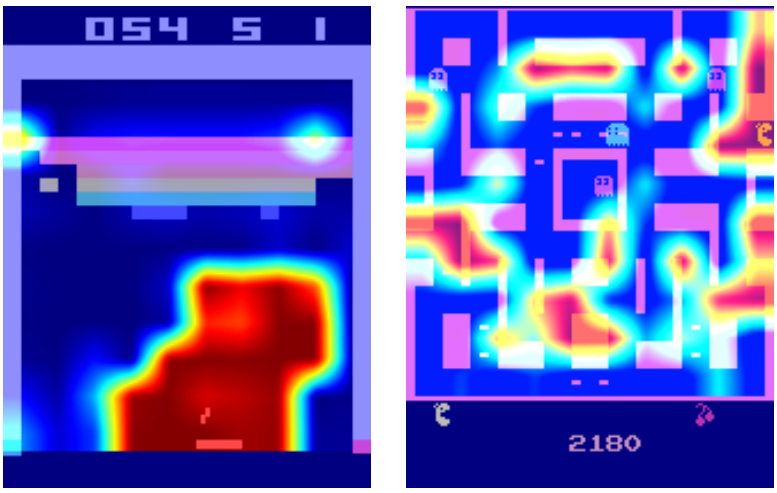
Three families of explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

Observation Perturbation

Perturbation analysis consists in applying various **modifications** to the observation in order to study their **impact** on the output of the **policy function** [Greydanus et al., 2018]. The greater the variation in the output caused by a perturbation applied to a given component of the input vector, the higher the **weight** assigned to that component. The magnitude of the perturbation is measured by computing the distance between the original output and the output obtained from the perturbed input. Using this set of weights, a *heatmap* is constructed, enabling the observer to visualize which parts of the observation vector are most sensitive in the action-selection process (see Figure 2.3a).

Observation Gradient Analysis

Observation **gradient** analysis refers to a set of methods that require a differentiable policy function in order to extract information from it [Simonyan et al., 2014, Weitkamp et al., 2019, Huber et al., 2019]. To this end, the gradient of the policy function is computed with respect to all **components of the input vector**. For each input component, the resulting gradient provides a weighting that represents its importance in the output of the policy function. Based on these weightings, a *heatmap* is constructed to allow the observer to visually identify the parts of the input vector that are the most influential in the policy function's



(a) (b)

Figure 2.4: Attention-based heatmaps obtained by [Mott et al., 2019] for the Atari games "Breakout" and "Ms. Pac-Man" (see Section 2.3.2). In (a), the highlighted regions are easily interpretable, whereas in (b), the attention is spread across many regions of the observation, making the explanation much harder to interpret.

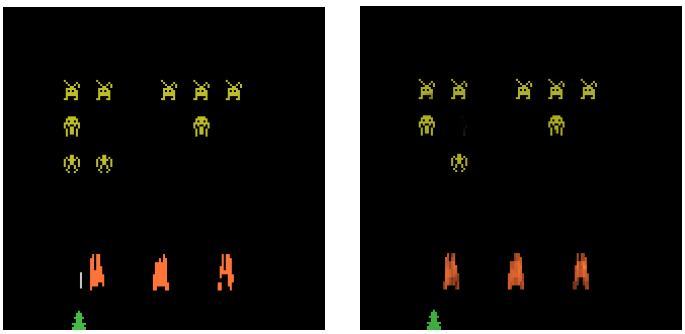
output (see Figure 2.3b).

Attention-Based Architecture

Attention blocks constitute a NN architecture that makes it possible to weight the relative importance of one part of a vector with respect to the other parts of the same vector [Vaswani et al., 2023]. These weights are computed as a function of the vector itself and are learned by the NN during training. For a given observation, each component of the observation vector is assigned a scalar value. It can therefore be assumed that the components associated with the highest weights are the most influential in determining the output [Mott et al., 2019, Itaya et al., 2021]. This attention-based *heatmap* is then provided to the observer as the explanation (see Figure 2.4a). However, as shown in Figure 2.4b, the attention weights are not always concentrated on a small, easily interpretable set of regions, which can make this type of explanation difficult to exploit.

2.3.3 Counterfactual

In the context of RL, counterfactual explanations are used to generate new observations. This new observation, distinct from the original one, leads the agent to adopt an alternative behavior. A counterfactual explanation aims to explain the agent's decision-making process. Modifying the observation informs the observer about the adjustments required for the agent to adopt a different behavior. The family of methods available to generate counterfactuals is the *use of the Latent Space (LS) for counterfactual generation.*



(a) (b)

Figure 2.5: Counterfactual explanation of the agent's behavior for the game *Space Invaders* [Olson et al., 2021]. (a) is the original observation, for which the agent selects the action "LEFT", whereas (b) is the counterfactual observation, for which the agent selects the action "RIGHT_FIRE".

Latent Space Usage for Counterfactual Generation

In this family of methods, **generative models** are used [Bond-Taylor et al., 2021] to exploit the Latent Representation (LR). This type of model learns to generate new data by imitating the training data distribution. In this context, generative models aim to produce alternative observations that are likely to alter the agent's actions (see Figure 2.5) [Olson et al., 2021].

The main challenge of this approach lies in the fact that, to serve as explanations, counterfactuals must be both **close** and **feasible**. A close counterfactual refers to one that does not differ significantly from the original observation. A feasible counterfactual corresponds to an observation that is meaningful within the considered environment.

2.3.4 Agent Trajectory

Until now, we have presented methods that focus on explaining individual decisions made by the agent. We now consider methods that aim to provide insight into the agent's strategy by analyzing its behavior over time.

The most direct approach consists in visualizing agent trajectories, most commonly through complete episodes. By analyzing these trajectories, the observer can infer the dynamics underlying the agent's behavior and how its decisions evolve through time. This type of visualization is not specific to XRL. In practice, visualizing agent episodes is a standard step in the development and evaluation of RL. It provides a simple way to verify that the agent behaves as expected and to detect potential implementation errors.

This simple approach can be further enhanced using families of methods such as *hierarchical decomposition* and *extraction via critic analysis.*

Extraction via Critic Analysis

In this family of methods, the predictions of the critic are used to determine which **trajectories to present to the observer** [Sequeira and Gervasio, 2020].

A set of states is selected from multiple trajectories. For a given state, all possible actions are examined. The state is assigned a value equal to the difference between the critic's prediction for the lowest-valued action and the prediction for the highest-valued action. Trajectories containing the states with the highest values according to this procedure are then chosen to be presented to the observer. These trajectories are considered **critical moments** of the agent and are therefore relevant for analysis. For this family of methods, the *selected trajectories* represent the interpretable information provided to the observer.

Hierarchical Decomposition

Hierarchical agents are composed of a set of **sub-policies** specialized for specific tasks within subsets of the environment's states. This family of methods distributes the optimization of the reward function across multiple models, significantly simplifying the learning process. Moreover, this **decomposition** of the policy function can also provide explainability in the agent's strategy [Beyret et al., 2019]. If a sub-policy is selected, it indicates that the agent will execute a specialized behavior. This behavioral specialization provides insight into the strategy the agent employs to maximize the reward function. The agent's *trajectory*, together with the *sub-policy* selected for each action, is used as interpretable information.

2.3.5 Trajectory Prediction

Making predictions about the future of a trajectory allows one to form a representation of what the system expects, based on its training experience. These predictions are produced using value functions. Such functions provide insights into the future evolution of the agent's trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent. Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$.

This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. To improve the interpretability of value-based explanations, a family of methods known as *critic enrichment* can be employed.

Critic Enrichment

In order to obtain interpretable information from the predictions of the **critic**, it is possible to **decompose** the critic into multiple **sub-functions** [Juozapaitis et al., 2019]. The reward function is thus decomposed into multiple sub-reward functions, each representing a **sub-objective** of the task to be accomplished. Similarly, for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics. Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize (see Figure 2.6). This *prediction* is then used as the explanation for the observer.

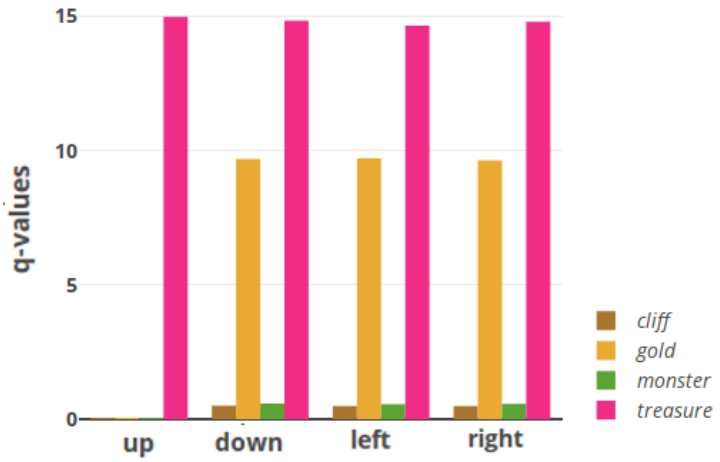


Figure 2.6: Prediction of sub-rewards as a function of the action selected by the agent for a given state [Juozapaitis et al., 2019]. Depending on the action chosen by the agent, one can determine which sub-reward it aims to maximize.

2.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent's decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent–environment interaction, this type of explanation aims to explain the agent's strategy. The families of methods used to obtain explanations in the form of agent–environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of **random variables** connected through a **Bayesian network** [Madumal et al., 2020]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables. At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer (see Figure 2.7). This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

Latent Space Usage for MDP Generation

Each layer of a NN can be viewed as a projection from one space to another [Zahavy et al., 2017]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent's

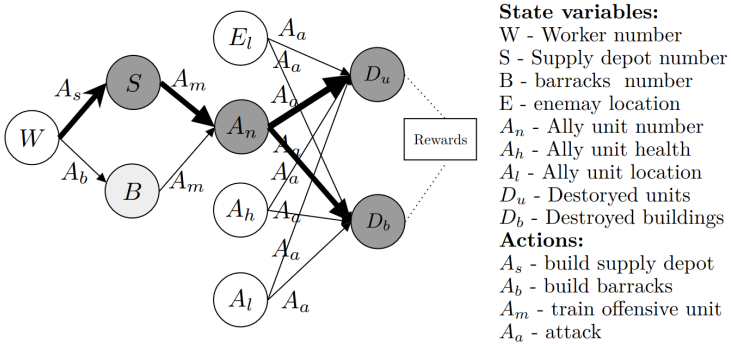


Figure 2.7: Agent–environment interaction model for the video game *StarCraft II* [Madumal et al., 2020]. Nodes represent random variables, and edges represent causal relationships between them.

perception. Based on this observation, it is possible to **redefine an MDP** by leveraging the **abstract representations** produced by the final layers of a NN. Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 2.8 illustrates this idea using a two-dimensional *t-SNE* projection of the LS of the NN used to represent the policy. Each point is colored according to the value function's prediction for the corresponding state. An agreement can be observed between the structure of this projected space and the critic's predictions: states that are assigned similar values by the critic also tend to be located close to one another in the projection. This agreement makes it possible to manually identify clusters of states, which likely correspond to states that are perceived as similar by the agent.

2.4 Conclusion

The taxonomy and the state-of-the-art review presented in this chapter make it possible to draw three main observations.

First, regarding the subject of explanation, Table 2.1 shows that every reviewed method targets either a **decision** or a **strategy**, and that none of them addresses the **learning process** itself, i.e., how the policy is progressively updated over the course of training. We identify two reasons for this absence. On the one hand, from a practical standpoint, this question is of limited interest: industrial applications generally deploy an already-trained policy, so the training procedure itself is rarely part of what needs to be explained once the agent is in production. On the other hand, training can itself be described as an iterative improvement of the policy. Explaining the learning process would therefore amount to explaining the sequence of policies produced across training iterations, which is conceptually close to explaining a strategy, only applied to the space of successive policies rather than to the space of successive states. Under this view, explaining a learning process does not constitute a fundamentally distinct problem but rather a variant of strategy explanation, which may account for

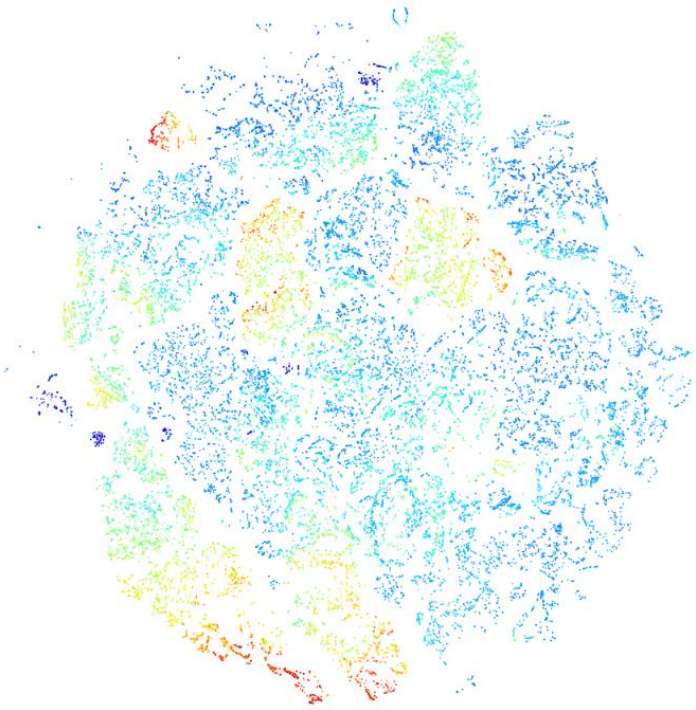


Figure 2.8: *t-SNE* projection of a neural network's latent space [Zahavy et al., 2017]. Each point corresponds to a state, and its color represents the value function's prediction for that state, with warmer colors indicating higher predicted values.

why it has not emerged as a separate subject in the literature.

Second, the diversity of explanation forms identified in this review (Table 2.1) reveals a limitation shared by nearly all the presented families of methods: the absence of reliable metrics to assess the impact of an explanation on the observer. Most works instead rely on subjective satisfaction questionnaires administered to human participants. This evaluation protocol is not fully satisfactory, since self-reported satisfaction is subject to numerous biases and does not necessarily reflect an actual improvement of the observer's mental model, as required by Definition 2.1.4.

Third, among the methods reviewed, *latent space usage for MDP generation* (Section 2.3.6) appears, in our view, the most promising direction. This method establishes a direct bridge between the deep representations learned by the agent and the search for concepts investigated in Chapter 1, applied here to the explanation of reinforcement learning agents. Rather than abstracting away or replacing the underlying deep learning support, as substitutive approaches do (Section 2.1), it exploits this support directly to produce an explanation, in line with the intrinsic explainability perspective adopted throughout this manuscript. However, as noted in Section 2.3.6, this method still relies on an informal, largely manual analysis of the projected latent space to identify clusters of states, which limits its rigor and its ability to systematically produce reliable explanations.

The next chapter builds on this approach and introduces a method designed to address this limitation.

Bibliography

[Barredo Arrieta et al., 2020] Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F. (2020). Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information Fusion, 58:82–115. **Article**.

[Bellucci et al., 2021] Bellucci, M., Delestre, N., Malandain, N., and Zanni-Merk, C. (2021). Towards a terminology for a fully contextualized xai. Procedia Computer Science, 192:241–250. **Article**.

[Bengio et al., 2014] Bengio, Y., Courville, A., and Vincent, P. (2014). Representation learning: A review and new perspectives.

[Beyret et al., 2019] Beyret, B., Shafti, A., and Faisal, A. A. (2019). Dot-to-dot: Explainable hierarchical reinforcement learning for robotic manipulation. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE. **Article**.

[Bond-Taylor et al., 2021] Bond-Taylor, S., Leach, A., Long, Y., and Willcocks, C. G. (2021). Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. **Article**.

[Bricken et al., 2023] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread. **Web**.

[Burda et al., 2018] Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. (2018). Large-scale study of curiosity-driven learning.

[Chen et al., 2021] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al. (2021). Evaluating large language models trained on code. **Article**.

[Coates et al., 2011] Coates, A., Ng, A. Y., and Lee, H. (2011). An analysis of single-layer networks in unsupervised feature learning. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223.

[Doshi-Velez and Kim, 2017] Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. **Article**.

Compiled on 29/7/2026 at 14:28

[Elhage et al., 2022] Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., et al. (2022). Toy models of superposition. Transformer Circuits Thread. **Web**.

[Garreau and von Luxburg, 2020] Garreau, D. and von Luxburg, U. (2020). Explaining the explainer: A first theoretical analysis of lime. In Chiappa, S. and Calandra, R., editors, Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, volume 108 of Proceedings of Machine Learning Research, pages 1287–1296. PMLR. **Article**.

[Greydanus et al., 2018] Greydanus, S., Koul, A., Dodge, J., and Fern, A. (2018). Visualizing and understanding atari agents. **Article**. **Code**.

[Gunning and Aha, 2019] Gunning, D. and Aha, D. (2019). Darpa's explainable ai (xai) program. AI Magazine, 40(2):44–58. **Article**.

[Guo et al.,] Guo, X., Gao, L., Liu, X., and Yin, J. Improved deep embedded clustering with local structure preservation. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, pages 1753–1759. International Joint Conferences on Artificial Intelligence Organization.

[Haas et al., 2024] Haas, R., Huberman-Spiegelglas, I., Mulayoff, R., Graßhof, S., Brandt, S. S., and Michaeli, T. (2024). Discovering interpretable directions in the semantic latent space of diffusion models. In 2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG), pages 1–9. IEEE.

[He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering.

[Huber et al., 2019] Huber, T., Schiller, D., and André, E. (2019). Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In KI 2019: Advances in Artificial Intelligence: 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings 42, pages 188–202. Springer. **Article**.

[Itaya et al., 2021] Itaya, H., Hirakawa, T., Yamashita, T., Fujiyoshi, H., and Sugiura, K. (2021). Visual explanation using attention mechanism in actor-critic-based deep reinforcement learning. **Article**.

[Jermyn et al., 2024] Jermyn, A., Templeton, A., Batson, J., and Bricken, T. (2024). Tanh penalty in dictionary learning. **Web**.

[Juozapaitis et al., 2019] Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., and Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In IJCAI/ECAI Workshop on explainable artificial intelligence. **Article**.

[Karvonen, 2024] Karvonen, A. (2024). Emergent world models and latent variable estimation in chess-playing language models.

[Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.

[Li et al., 2024] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. (2024). Emergent world representations: Exploring a sequence model trained on a synthetic task.

[Lu et al., 2019] Lu, G., Ouyang, W., Xu, D., Zhang, X., Cai, C., and Gao, Z. (2019). Dvc: An end-to-end deep video compression framework.

[MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings ofthe 5th Berkeley Symposium on Mathematical Statisticsand Probability, volume 1, pages 281–297. **Article**.

[Madumal et al., 2020] Madumal, P., Miller, T., Sonenberg, L., and Vetere, F. (2020). Explainable reinforcement learning through a causal lens. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 2493–2500. **Article**.

[Marconato et al., 2023] Marconato, E., Passerini, A., and Teso, S. (2023). Interpretability is in the mind of the beholder: A causal framework for human-interpretable representation learning.

[Melamed and Lin, 2023] Melamed, Y. Y. and Lin, M. (2023). Principle of Sufficient Reason. In Zalta, E. N. and Nodelman, U., editors, The Stanford Encyclopedia of Philosophy. Metaphysics Research Lab, Stanford University, Summer 2023 edition.

[Miklautz et al.,] Miklautz, L., Klein, T., Sidak, K., Leiber, C., Lang, T., Shkabrii, A., Tschiatschek, S., and Plant, C. Breaking the reclustering barrier in centroid-based deep clustering.

[Miller, 2019] Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. Artificial Intelligence, 267:1–38. **Article**.

[Misak, 2013] Misak, C. (2013). The American Pragmatists. Oxford University Press.

[Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

[Mott et al., 2019] Mott, A., Zoran, D., Chrzanowski, M., Wierstra, D., and Jimenez Rezende, D. (2019). Towards interpretable reinforcement learning using attention augmented agents. Advances in neural information processing systems, 32. **Article**.

[Newell, 1980] Newell, A. (1980). Physical symbol systems. Cognitive Science, 4(2):135–183.

[Olah et al., 2020] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020). Zoom in: An introduction to circuits. Distill. **Article**.

[Olson et al., 2021] Olson, M. L., Khanna, R., Neal, L., Li, F., and Wong, W.-K. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. Artificial Intelligence, 295:103455. **Article**. **Code**.

[Payani and Fekri, 2019] Payani, A. and Fekri, F. (2019). Inductive logic programming via differentiable deep neural logic networks. **Article**. **Code**.

[Peirce, 1878] Peirce, C. S. (1878). How to make our ideas clear. Popular Science Monthly, 12:286–302.

[Radford et al., 2021] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision.

[Ren et al.,] Ren, Y., Pu, J., Yang, Z., Xu, J., Li, G., Pu, X., Yu, P. S., and He, L. Deep clustering: A comprehensive survey.

[Rudin, 2019] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1(5):206–215. **Article**.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Sequeira and Gervasio, 2020] Sequeira, P. and Gervasio, M. (2020). Interestingness elements for explainable reinforcement learning: Understanding agents' capabilities and limitations. Artificial Intelligence, 288:103367. **Article**.

[Seshia et al., 2022] Seshia, S. A., Sadigh, D., and Sastry, S. S. (2022). Toward verified artificial intelligence. Communications of the ACM, 65(7):46–55. **Article**.

[Sieusahai and Guzdial, 2021] Sieusahai, A. and Guzdial, M. (2021). Explaining deep reinforcement learning agents in the atari domain through a surrogate model. **Article**.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Simon, 1996] Simon, H. A. (1996). The Sciences of the Artificial. MIT Press, 3 edition.

[Simonyan et al., 2014] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. **Article**. **Code**.

[Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread. **Web**.

[Topin et al., 2021] Topin, N., Milani, S., Fang, F., and Veloso, M. (2021). Iterative bounding mdps: Learning interpretable policies via non-interpretable methods. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 35, pages 9923–9931. **Article**.

[Trivedi et al., 2022] Trivedi, D., Zhang, J., Sun, S.-H., and Lim, J. J. (2022). Learning to synthesize programs as interpretable and generalizable policies. **Article**. **Code**.

[Vaswani et al., 2023] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention is all you need. **Article**.

[Voynov and Babenko, 2020] Voynov, A. and Babenko, A. (2020). Unsupervised discovery of interpretable directions in the gan latent space.

[Wei et al.,] Wei, X., Zhang, Z., Huang, H., and Zhou, Y. An overview on deep clustering. 590:127761.

[Weitkamp et al., 2019] Weitkamp, L., van der Pol, E., and Akata, Z. (2019). Visual rationalizations in deep reinforcement learning for atari games. **Article**.

[Xie et al.,] Xie, J., Girshick, R., and Farhadi, A. Unsupervised deep embedding for clustering analysis.

[Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying the black box: Understanding dqns. **Article**.